



میزکار / درس‌های من / برنامه‌سازی پیشرفته / عمومی / آزمون میان‌ترم

شروع سه‌شنبه، 24 فروردین 1400، 10:31 صبح

وضعیت پایان یافته

پایان سه‌شنبه، 24 فروردین 1400، 11:14 صبح

زمان صرف شده 43 دقیقه 52 ثانیه

نمره 17.00 از 19.00 (89%)

سؤال 1

درست

نمره 1.00 از 1.00

کدام یک از موارد زیر از مزایای طراحی بالا به پایین محسوب نمی‌شود؟



a. بالا رفتن سرعت اجرای برنامه ☒

b. ساده‌شدن آزمون برنامه ☐

c. بالا رفتن خوانایی برنامه ☐

d. ساده‌شدن طراحی برنامه ☐

پاسخ درست »

بالا رفتن سرعت اجرای برنامه است.

تابع بازگشتی زیر قرار است دو بردار مرتب (غیرنزولی) از اعداد را با یکدیگر ادغام کند و یک بردار مرتب متشکل از عناصر هر دوی آنها در پارامتر result برگرداند. کدام قطعه کد بخش /* ? */ را به درستی کامل می‌کند؟

```
void merge(vector<int> x, vector<int> y, vector<int>& result, int i = 0, int j = 0) {
    if (i == x.size() || j == y.size()) {
        /* ? */
    }
    if (x[i] < y[j]) {
        result.push_back(x[i]);
        merge(x, y, result, i+1, j);
    } else {
        result.push_back(y[j]);
        merge(x, y, result, i, j+1);
    }
}

/* 1 */
for (int j2 = j; j2 < y.size(); j2++) result.push_back(y[j2]);
for (int i2 = i; i2 < x.size(); i2++) result.push_back(x[i2]);
return;

/* 2 */
for (int j2 = j; j2 < y.size(); j2++) result.push_back(y[j2]);
for (int i2 = i; i2 < x.size(); i2++) result.push_back(x[i2]);

/* 3 */
return;

/* 4 */
for (int k = i+j; k < x.size() + y.size(); k++)
    result.push_back(x[k] < y[k] ? x[k] : y[k]);
return;
```

1.a ☒3.b ☐4.c ☐2.d ☐

پاسخ درست »
1 « است.

بدنه تابع main یک برنامه شامل خطوط زیر است:

```
string s;  
int cnt = 0;  
while (cin >> s)  
    cnt++;
```

اگر ورودی زیر را به این برنامه بدهیم مقدار cnt چیست؟ (شماره‌های سمت چپ نشان‌دهنده شماره خط هستند و جزء ورودی محسوب نمی‌شوند).

```
1 Hello, Peter!  
2  
3  
4 OMG!
```

a. 20 ☐

b. 3 ☒

c. 6 ☐

d. 4 ☐

پاسخ درست »

3 است.



کدام یک از خط‌های مشخص شده با کامنت‌های ۱ تا ۶ خطای کامپایل دارند؟

```
void f(int& a) { a--; }

void passByRef(int b, vector<int> v, int& c)
{
    int i = c;
    f(i);          // 1
    f(b);          // 2
    f(&b);         // 3
    f(c);          // 4
    f(v[i]);       // 5
    f(v[b-1]);     // 6
    f(v[b]-1);     // 7
}
```

☒ a. ۳ و ۷

☐ b. ۲ و ۴ و ۶

☐ c. ۱ و ۲ و ۷

☐ d. ۳ و ۴ و ۵

پاسخ درست »

۳ و ۷ است.

کدام یک از موارد از مزیت‌های استفاده از تایپ string نسبت به کار با آرایه‌ای از کارکترها نیست؟

☐ a. خوانایی برنامه با استفاده از string نسبتاً بالاتر است

☐ b. احتمال ارتکاب خطای برنامه‌نویسی در استفاده از string کمتر است

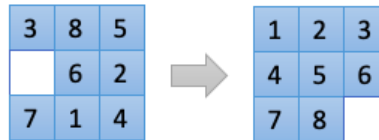
☐ c. مدیریت حافظه در string به طور خودکار انجام می‌شود

☒ d. متدهای string پیاده‌سازی کارتری نسبت به توابع مشابه در string.h دارند

پاسخ درست »

متدهای string پیاده‌سازی کارتری نسبت به توابع مشابه در string.h دارند است.

در مسئله معمای ۸، در یک صفحه ۸ مربع متحرک هستند که با اعداد ۱ تا ۸ شماره‌گذاری شده‌اند و یک مربع خالی نیز وجود دارد. بازیکن در هر حرکت یکی از مربع‌های مجاور خانه خالی را به خانه خالی منتقل می‌کند. هدف مسئله مرتب کردن مربع‌ها مطابق وضعیت سمت راست در شکل زیر است:



در شبه کد زیر یک راه‌حل مبتنی بر backtracking برای این مسئله ارائه شده است. مشکل این راه‌حل چیست؟

تابع حل مسئله:

اگر صفحه در وضعیت نهایی است:

نتیجه مثبت برگردان

در غیر این صورت:

به ازای هر یک از چهار جهت بالا، پایین، چپ، راست:

اگر خانه خالی در جهت مربوطه همسایه دارد:

همسایه را به خانه خالی منتقل کن

تابع را به طور بازگشتی فراخوانی کن

اگر نتیجه فراخوانی مثبت است: نتیجه مثبت برگردان

نتیجه منفی برگردان

a. شبه‌کد پیشنهادی مبتنی بر روش backtracking نیست

b. این الگوریتم قابل پیاده‌سازی در زبان سی‌پلاس‌پلاس نیست

c. ممکن است اجرای تابع هیچ‌وقت خاتمه پیدا نکند

d. ممکن است تابع نتیجه نادرستی تولید کند



پاسخ درست «ممکن است اجرای تابع هیچ‌وقت خاتمه پیدا نکند» است.

کدام یک از قطعه‌کدهای داده‌شده به درستی بردار v را مانند شکل زیر مقداردهی می‌کند؟

3	4	5
0		
0	12	

- 1 `vector<int> v(3,3);
v[0][0] = 3; v[0][1] = 4; v[0][2] = 5;
v[2][1] = 12;`
- 2 `vector<vector<int>> v(3,3);
v[0][0] = 3; v[0][1] = 4; v[0][2] = 5;
v[2,1] = 12;`
- 3 `vector<int> v;
v.push_back({3, 4, 5});
v.push_back({0});
v.push_back({0, 12});`
- 4 `vector<int> w(1);
vector<vector<int>> v(3,w);
v[0][0] = 3; v[0].push_back(4); v[0].push_back(5);
v[2].push_back(12);`

a. ☐ 1

b. ☐ 2

c. ☒ 4

d. ☐ 3

پاسخ درست « 4 » است.



کدام یک از جملها در مورد مقایسه رد کردن پارامتر با مقدار (by value) و با ارجاع (by reference) صحیح نیست؟

- ☐ a. رد کردن با ارجاع می‌تواند باعث صرفه‌جویی در مصرف حافظه شود
- ☐ b. برای پارامترهایی با اندازه بزرگ رد کردن با ارجاع سربار زمانی فراخوانی تابع را کمتر می‌کند
- ☐ c. دسترسی به پارامتری که با ارجاع رد شده کندتر است
- ☒ d. رد کردن با مقدار خوانایی کمتری نسبت به رد کردن با ارجاع دارد

پاسخ درست »

رد کردن با مقدار خوانایی کمتری نسبت به رد کردن با ارجاع دارد» است.

کدام تعریف برای تابعی که قرار نیست تغییری در محتوای بردار بدهد مناسب‌تر است؟

- 1 `void process_vector(vector<int> v);`
- 2 `void process_vector(vector<int>& v);`
- 3 `void process_vector(const vector<int> v);`
- 4 `void process_vector(const vector<int>& v);`

2. a ☐

3. b ☐

1. c ☐

4. d ☒

پاسخ درست »

4» است.

سؤال 10

نادرست

نمره 0.00 از 1.00

در محاسبات بازگشتی، یک مسئله را به زیرمسئله(های) از همان نوع اما کوچکتر تبدیل می‌کنیم. در برنامه سفر اسب، «کوچکتر» بودن زیر مسئله چه تعبیری دارد؟

☐ a. تعداد سطرهای باقی‌مانده صفحه

✖

☒ b. تعداد انتخاب‌های ممکن اسب برای حرکت بلافاصله بعد

☐ c. تعداد خانه‌های ملاقات نشده توسط اسب

☐ d. ابعاد صفحه شطرنج

پاسخ درست »

تعداد خانه‌های ملاقات نشده توسط اسب» است.

سؤال 11

درست

نمره 1.00 از 1.00

کدام‌یک از موارد زیر از مشکلات متغیرهای سراسری نیست؟

☐ a. سخت‌شدن استفاده مجدد

✔

☒ b. سرعت پایین اجرای برنامه

☐ c. خوانایی پایین برنامه

☐ d. خط‌پذیری بالای برنامه

پاسخ درست »

سرعت پایین اجرای برنامه» است.

سؤال 12

درست

نمره 1.00 از 1.00

کدام یک از گزینه‌ها صحیح نیست؟

a. وابسته کردن برنامه به فرمت ورودی و خروجی می‌تواند سرعت تولید برنامه را بالا ببرد

b. استفاده از توابع موجود در کتابخانه‌های آماده سرعت تولید برنامه را بالا می‌برد



c. سپردن یک وظیفه به هر تابع باعث افزایش سرعت اجرای برنامه می‌شود

d. در کوتاه‌مدت، نوشتن توابع و کلاس‌های قابل استفاده مجدد می‌تواند سرعت تولید برنامه را پایین بیاورد

پاسخ درست »

سپردن یک وظیفه به هر تابع باعث افزایش سرعت اجرای برنامه می‌شود» است.

سؤال 13

کامل

نمره داده نشده

تابع زیر ترکیب r از n را محاسبه می‌کند. چرا اجرای این تابع برای مقادیر بزرگ طول می‌کشد؟

a. فراخوانی‌های تکراری

b. سربار زمانی فراخوانی‌های بازگشتی

c. کندی محاسبات با اعداد بزرگ

d. سربار مدیریت حافظه پشته (stack)

پاسخ درست »

فراخوانی‌های تکراری» است.

در این تابع که قرار است کلماتی که بلافاصله تکرار می‌شوند را در خروجی بنویسد پس از ورود دو کلمه یکسان در ترمینال، چه زمانی خروجی repeated word ... نوشته می‌شود؟

```
int main()
{
    string previous = "";
    string current;
    while (cin >> current) {
        if (previous == current)
            cout << "repeated word: " << current << '\n';
        previous = current;
    }
}
```



a. پس از ورود کامل سطری که شامل دو کلمه یکسان است ☒

b. پس از اتمام کل ورودی ☐

c. پس از ورود اولین فاصله خالی بعد از دو کلمه یکسان ☐

d. بلافاصله بعد از ورود دو کلمه یکسان ☐

پاسخ درست »

پس از ورود کامل سطری که شامل دو کلمه یکسان است» است.

با فرض این که در بردار h یک اندیس i وجود دارد که عناصر قبل از آن یک دنباله اکیداً صعودی و عناصر بعد از آن یک دنباله اکیداً نزولی را تشکیل می‌دهند (مطابق شرح پرسش ۱ از تکلیف دوم درس)، تابع زیر به سبک جستجوی دودویی آن اندیس را پیدا می‌کند. در فراخوانی اول، مقدار $from$ صفر و مقدار to برابر $h.size()-1$ فرض می‌شود. کدامیک از گزینه‌ها به درستی بخش `/* ? */` را کامل می‌کند؟

```
int find(vector<int> h, int from, int to) {
    if (from == to)
        return from;
    int mid = (from + to) / 2;
    /* ? */
    if (left > h[mid] && h[mid] > right)
        return find(h, from, mid - 1);
    if (left < h[mid] && h[mid] < right)
        return find(h, mid + 1, to);
    return mid;
}

/* 1 */
int left = h[mid] - 1;
int right = h[mid] + 1;

/* 2 */
int left = h[mid-1];
int right = h[mid + 1];

/* 3 */
int left = mid > 0 ? h[mid-1] : h[mid] - 1;
int right = mid < h.size()-1 ? h[mid + 1] : h[mid] + 1;

/* 4 */
int left = mid == from ? h[mid-1] : h[mid] - 1;
int right = mid == to ? h[mid + 1] : h[mid] + 1;
```

4.a ☐2.b ☒1.c ☐3.d ☐

پاسخ درست »
3 است.



خروجی فراخوانی `g("", s)` چیست؟

```
void g(string pre, string post) {
    if (post == "")
        return;
    cout << pre << "?" << post.substr(1) << endl;
    g(pre + post[0], post.substr(1));
}
```



- ☐ a. رشته‌هایی که در هر کدام، یکی از کاراکترهای `s` با ؟ جایگزین شده است
- ☐ b. رشته‌هایی که در هر کدام، یک کاراکتر ؟ در یک محل متفاوت در `s` درج شده است
- ☐ c. رشته‌هایی که در هر کدام، یک پسوند از `s` با ؟ جایگزین شده است
- ☐ d. رشته‌هایی که در هر کدام، یک پیشوند از `s` با ؟ جایگزین شده است

پاسخ درست «رشته‌هایی که در هر کدام، یکی از کاراکترهای `s` با ؟ جایگزین شده است» است.

کدام یک از خطوط زیر تمام عناصر بردار `v` (با عناصر `int`) را می‌نویسد؟

```
1 cout << v;
2 for (auto n : v) cout << n;
3 for (auto n : v) cout << v[n];
4 for (vector<int> n : v) cout << v[n];
```



- ☐ 4. a
- ☐ 3. b
- ☒ 2. c
- ☐ 1. d

پاسخ درست «2» است.

مزیت مهم توابع بازگشتی چیست؟



a. ساده‌تر کردن استدلال درباره درستی عملکرد تابع ☐

b. قابلیت استفاده مجدد بالاتر تابع ☐

c. مصرف کمتر حافظه نسبت به توابع غیربازگشتی ☐

d. سرعت بیشتر اجرای تابع ☐

پاسخ درست »

ساده‌تر کردن استدلال درباره درستی عملکرد تابع « است.

بخش مشخص شده با `/* missing code */` را با چه گزینه‌ای جایگزین کنیم تا رابطه زیر برقرار باشد؟

$$f(x, n) = \sum_{i=1}^n \frac{x^i}{i!}$$

```
double f(double x, int n, int i = 1, double term = 1) {
    if (i > n)
        return 0;
    return /* missing code */;
}
```

a. `x * f(x, n, i + 1, term + x / i)` ☐

b. `term * f(x, n, i + 1, term * x / i)` ☐

c. `(term + f(x, n, i + 1, term * x / i` ☒

d. `x * f(x, n, i + 1, term / i)` ☐

پاسخ درست »

`(term + f(x, n, i + 1, term * x / i`

« است.

تابع زیر چه مشکلی دارد؟

```
void check_sorted(const vector<int>& numbers) {  
    bool not_sorted = false;  
    for (int i = 0; i < numbers.size() - 1; i++)  
        if (numbers[i] > numbers[i+1]) {  
            cout << "Not sorted\n";  
            not_sorted = true;  
        }  
    if (!not_sorted)  
        cout << "Sorted\n";  
}
```



a. تلفیق دو وظیفه متفاوت ☒

b. امکان تغییر در بردار numbers ☐

c. استفاده از متغیرهای سراسری ☐

d. پایین بودن سرعت اجرا ☐

پاسخ درست »

تلفیق دو وظیفه متفاوت» است.